

How To Optimize GEMM and Conv (List of all Optimization + Missing Optimizations in MLIR RL)

Below is a “compiler engineer” checklist of what it usually takes to get **PyTorch / oneDNN-class CPU performance** for **GEMM**, **Conv**, and simple **elementwise add**, and then a concrete mapping to **what your current repo/action space already has** vs **what’s missing**, with **how to implement each missing piece in MLIR** (preferably with **Linalg + Transform dialect**).

Your own paper already hints at the core issue: **MatMul and Conv lag PyTorch because PyTorch is using highly specialized kernels (register tiling, aggressive vectorization, etc.) that your current action space doesn’t expose**, and your vectorization is currently constrained because the current vectorization approach can generate **very large vectors / full unrolling**, so you mask it unless the (tiled) loop sizes are small.

1) What experts typically do for peak CPU performance

Think of this as a 3-level stack:

- **Thread-level (parallelism)**: how work is split across cores
- **Cache-level (blocking/packing)**: how data is staged for L3/L2/L1
- **Register/SIMD-level (microkernel)**: how the inner kernel maps to vector registers & FMA

1.1 Optimizations that matter for all of GEMM / Conv / Add

Loop & iteration-space transforms

- Multi-level tiling (cache tiling + register tiling).
- Loop interchange to make inner loops unit-stride and vector-friendly.
- Strip-mining + unroll-and-jam (explicit unroll factors).
- Loop peeling / remainder separation (full tiles vs partial tiles).
- Loop distribution (sometimes needed to isolate vectorizable pieces).
- LICM / invariant hoisting (you already do some of this).

Vectorization & tail handling

- Vectorization with *explicit* control over:
 - vector widths / vector shapes,
 - lowering strategy (outerproduct/dot/matrix intrinsics),

- tail strategy: peeling, padding, masking.
- Full/partial separation:
 - either **peel** remainder loops, OR
 - **pad** to multiples of vector widths, OR
 - use **masked vector transfers**.
- Align loads/stores; avoid misaligned penalties.

Memory hierarchy

- **Packing** (a.k.a. “copy to a blocked layout”) of one or both operands.
- **Promotion** of tiles to fast memory (stack / alloca / scratchpad) to improve locality.
- Bufferization strategy that avoids repeated allocations & copies.

Parallel runtime

- Thread count selection policy (size-dependent).
- Thread affinity / pinning.
- Scheduling policy (static vs dynamic, chunk sizes).
- Avoid parallelizing when the tensor is too small (parallel overhead dominates).

Math/codegen flags

- Fast-math / FMA contraction (when acceptable for your correctness budget).
- Target CPU features enabled (AVX2/AVX-512/AMX etc).
- Prefetching / pipelining (optional but can matter).

The Linalg dialect itself explicitly calls out many of these as key transformations (tiling, promotion, fusion, vectorization, mapping to parallel loops, and lowering to library calls or special instructions) ([MLIR](#)).

2) GEMM: full “expert” checklist (CPU)

Your list is already strong; the key missing “expert kernel” pieces are **packing**, **promotion**, and **microkernel/register tiling** with controlled unrolling.

2.1 GEMM optimizations that are usually required

I’ll group these in the order you’d typically build a schedule.

A) Thread-level parallelism (outermost)

- Parallelize over **M** and/or **N** blocks (not K unless using split-K reduction).

- Choose thread count based on problem size (avoid oversubscription; avoid tiny parallel loops).
- Static scheduling + affinity (for stable perf).

B) Cache tiling (L3/L2/L1)

- 2–3 levels of tiling over (M, N, K):
 - e.g., **(MC, NC, KC)** for L2/L3,
 - then **(M1, N1, K1)** for L1,
 - then **(MR, NR, KR)** for registers.
- Loop interchange so:
 - K is in a good position for reuse,
 - the innermost dimension(s) are unit-stride for vector loads.

C) Packing (array blocking) + promotion

This is the biggest “PyTorch gap” for GEMM:

- **Pack B** (and often A) into a **blocked contiguous layout** matching the microkernel shape.
- **Promote** packed tiles to stack/scratch to keep them hot in cache and enable predictable accesses.
- Ensure alignment (32/64 bytes) and pad to vector multiples if needed.

D) Register tiling / microkernel

- Choose **MR×NR** tile sizes that match ISA (e.g., 8×8 / 6×16 / etc).
- Keep the **C tile in registers** (vector SSA values), reduce into registers.
- Unroll over K (KR factor), often with unroll-and-jam.

E) Tail strategy

- Peel remainder loops OR pad input tiles OR use masked vector transfers.
- Generate separate “full-tile” fast path and “remainder” slow path.

F) Epilogue fusion (if applicable)

- Fuse bias add, activation, clamp, etc. into the GEMM store path.

G) Additional (often last 5–10%)

- Prefetch distance tuning.
 - Software pipelining for the microkernel loops.
 - L1/L2 prefetch or streaming stores depending on store reuse.
-

3) Conv: full “expert” checklist (CPU)

Convolution is harder because **algorithm choice + layout choice** dominates.

3.1 Conv optimizations that are usually required

A) Pick the algorithm (shape-dependent)

- **Direct convolution** (blocked + vectorized): common for many shapes.
- **im2col + GEMM**: can be good, but may be limited by the cost of materializing/im2col and by vectorization of the packing kernel.
- **Winograd** (esp 3×3) or other transforms for specific cases.
- Depthwise conv has its own kernels.

B) Layout transforms (critical)

- Choose a layout that vectorizes well:
 - NHWC/HWCF is common for SIMD-friendly inner channel loops.
 - Or blocked formats like `...16c` (oneDNN style) for AVX-512.
- Potentially transform:
 - NCHW ↔ NHWC (channels-last)
 - HWCF/FHWC permutations for filters (your code already touches this idea).
- Block the channel dimension to match vector width (C-blocking / K-blocking).

C) Tiling (spatial + channel)

- Parallelize over N / OH / OW / OC blocks (depending on shape).
- Tile OH/OW spatially for cache.
- Tile IC/OC for SIMD, and for weight reuse.

D) If using im2col

- The im2col “packing” kernel must be vectorizable; otherwise it becomes the bottleneck.
- Prefer **fusing** im2col with GEMM or doing “implicit im2col”/packing so you don’t materialize a huge temporary.
- Use the **same GEMM microkernel pipeline** after lowering to matmul.

This is not hypothetical: even in IREE/MLIR land, people observe im2col tensor kernels can become inefficient and hard to vectorize because of complicated indexing maps, and specialized vectorization patterns may be needed ([GitHub](#)).

E) Direct conv vectorization & tiling in MLIR

Upstream MLIR has explicitly worked on:

- Vectorization support for `linalg.conv_2d_nhwc_hwcf` (reviews.llvm.org)
- Convolution tiling leveraging matmul-style tiling infrastructure (reviews.llvm.org)

So there *are* MLIR-native “expert recipes” for direct conv, but they typically require the same missing building blocks: tiling → remainder handling → vectorization → lowering.

4) Add (elementwise): what “perfect perf” usually means

Elementwise add is almost purely **memory bandwidth** bound.

To get near-peak:

- Ensure unit-stride iteration order (interchange if needed).
- Vectorize to the widest supported vector width.
- Unroll a bit (often small factor).
- Parallelize only when the tensor is large enough to amortize overhead.
- Thread affinity matters (and NUMA first-touch for huge buffers).
- Fusion is often the biggest win in real graphs (avoid materializing intermediate add results).

If add is underperforming, it’s commonly:

- parallel runtime overhead / wrong thread count,
 - no pinning / poor thread placement,
 - missed vectorization due to tails/alignment.
-

5) What your repo currently provides (actions/optimizations present)

From the code you attached, your RL **action space** is (folder `rl_autoschedular/actions/`):

1. **Tiling** (`tiling.py`)
 - Uses `transform.structured.tile_using_for`.
 - **Tile sizes are restricted to powers of two:** `ts = 2^(param-1)`.
 - **Only tile sizes that exactly divide the loop upper bound are allowed** (`ub % ts == 0`) and `ts != ub`.
 - This means: **no remainder loops exist**, but also **you cannot pick many good tile sizes** (e.g., 6, 12, 24, 48, etc.) and you can’t handle prime-ish bounds.

2. **Tiled Parallelization** (`tiled_parallelization.py`)
 - Uses `transform.structured.tile_using_forall` on parallel iterators (then tiles reductions with `tile_using_for`).
 - Relies on later lowering: `scf.forall` → `scf.parallel` → OpenMP (your `execution.py` pass pipeline includes `scf-forall-to-parallel`, `convert-scf-to-openmp`).
 - You don't expose:
 - explicit thread count,
 - schedule kind/chunk,
 - affinity.

This matches the paper's description of parallelization being lowered to OpenMP worksharing loops.

3. **Tiled Fusion** (`tiled_fusion.py`)
 - Uses `transform.structured.tile_and_fuse` to fuse producers into a consumer.
4. **Interchange** (`interchange.py`)
 - Uses `transform.structured.interchange` (after `transform.structured.generalize`).
 - Candidate generation may be restricted depending on config (`ENUMERATED` vs `POINTER`).
5. **Vectorization** (`vectorization.py`)
 - Uses `transform.structured.vectorize` plus a set of vector lowering patterns (via your `transform_bufferize_and_lower_v` pipeline).
 - It is **terminal**.
 - It is guarded by a size heuristic (`vect_size_limit`, default 512) to avoid generating huge vectors / unrolled code (your paper explains this issue).
 - For conv/pooling it may do preprocessing (`transpose_conv2d`, `decompose`).
6. **No transformation.**

Important: conv is converted to im2col in your benchmark loader

In `rl_autoschedular/benchmarks.py`, you call `transform_img2col` on conv2d ops before feature extraction. So in this repo version, “conv performance” is largely “img2col packing kernel + matmul performance”.

6) What you are missing vs an “expert” GEMM/Conv/Add recipe

Here's the most important part: a comprehensive “missing list” with concrete MLIR ways to implement each.

6.1 Missing for GEMM (most impactful first)

(1) Packing / blocked layouts (array blocking)

Why it matters:

- PyTorch/oneDNN GEMM performance heavily depends on **packing** (especially B) to create unit-stride, cache-friendly, SIMD-friendly access patterns.

How to do it in MLIR:

- Use Transform dialect **pack** ops:
 - `transform.structured.pack` (lets you specify packed sizes and padding)
 - `transform.structured.pack_greedily` (auto-picks a packing strategy for matmul-like subcomputations; initial implementation uses sizes like `[32, 8, 16]` and reorders loops for favorable access)
- After packing, you often need to **lower** pack/unpack:
 - there is a `transform.structured.lower_pack` pipeline used in practice (see the pack/unpack/mmt4d recipe example)

Practical RL integration:

- Add a new action: **PackMatmul** with parameters:
 - pack A? pack B?
 - packed_sizes for (M,N,K) (do not restrict to powers-of-two)
 - optional `pack_transpose` for B
 - padding strategy (0/identity) for tails.
- Or add a “macro-action” that runs `pack_greedily` before the normal tiling/vectorization.

(2) Promotion to fast buffers (scratchpad / stack)

Why it matters:

- Helps keep tiles hot in cache and can eliminate repeated memory traffic.
- Also helps “full tile vs partial tile” handling by allocating **full tile buffers** and using subviews for remainders.

MLIR support:

- Transform dialect has `transform.structured.promote` with knobs:
 - `operands_to_promote`

- `use_full_tiles_by_default`
 - `use_full_tile_buffers`
 - `alignment`
- Under the hood, promotion is built around the “full tile buffer + partial slice + copy” strategy described in Linalg transforms (promoteSubViews) ([MLIR](#)).

Practical RL integration:

- Add a new action: **PromoteOperands** with parameters:
 - which operands (A,B,C) to promote
 - full-tiles-only vs allow partial
 - alignment (32/64)
 - memory space choice (stack vs heap; usually stack/alloca for small tiles).

(3) Register tiling / microkernel shaping

Why it matters:

- This is the “MR×NR” microkernel that gives oneDNN its core speed.
- It’s not just “vectorize”; it’s **vectorize after shaping the inner loops to match the ISA**.

In MLIR terms, you want:

- inner tile sizes (MR, NR, KR) that become vector shapes / contraction sizes,
- vectorization producing `vector.contract` / `vector.fma` sequences,
- contraction lowering strategy tuned (outerproduct/dot/intrinsics).

How to implement:

- This can be done with **an explicit inner tiling level** that targets vector register shapes, followed by `transform.structured.vectorize` and your existing vector lowering pipeline.
- Your current framework can do “multi-level tiling” only by repeating tiling actions, but you’re limited by episode length and by tile candidate constraints. A better approach is:
 - add a **single macro-action**: “TileToCacheThenTileToRegisters”.
- If you want a known-good recipe to copy, IREE’s matmul path uses **mmt4d + packing + microkernels** as a core strategy .

(4) Explicit unroll factor / unroll-and-jam

Why it matters:

- You need to tune K-unroll (KR) and sometimes unroll-and-jam for throughput.

You currently have only a “full unroll vs no unroll” flavor (and you avoid vectorization when it would blow up).

MLIR support:

- Transform dialect has SCF loop transforms including:
 - `transform.loop.unroll` (explicit factor),
 - `transform.loop.unroll_and_jam`,
 - `transform.loop.peel`

Practical RL integration:

- Add an action: **UnrollInnerLoops(factor)** applied after tiling (match the innermost `scf.for` loops).
- Add an action: **UnrollAndJam(factor)** for the loop immediately outside the reduction.

(5) Remainder handling: peel/pad/mask

Why it matters:

- If you want optimal tile sizes (not necessarily divisors of the dimension), you must handle remainder tiles.
- Your current action space avoids tails by only allowing divisors, which is a big performance and expressivity limitation.

MLIR support options:

- **Peeling:** `transform.loop.peel`
- **Padding:**
 - packing ops support padding values (pack can create padded blocks)
 - Linalg also has pad vectorization patterns (`populatePadOpVectorizationPatterns`) ([MLIR](#))
- **Masked vectorization:**
 - You already apply `vector.split_transfer_full_partial` and `vector.transfer_to_scf`, which is part of the “full/partial” strategy for vector transfers (good), but you need loop-level remainder strategies too.

Practical RL integration:

- Relax tiling candidate generation:
 - allow non-divisors, allow tiles > ub (clamp), etc.
- Add a follow-up action:
 - **PeelRemainders** on selected loops OR **PadToMultiple** on selected dimensions.

6.2 Missing for Conv (beyond “just make matmul fast”)

Because your conv is (currently) converted to im2col in your benchmark harness, the main missing pieces are:

(1) Make the img2col packing kernel fast (or avoid materializing it)

- im2col kernels can be hard to vectorize due to complex indexing maps, and can dominate runtime ([GitHub](#)).
- Upstream MLIR's im2col rewrite has been evolving (e.g., producing more canonical `linalg.generic` forms) ([lists.llvm.org](#)).

MLIR implementation options:

- **Fuse** the img2col producer into the matmul consumer (your `TiledFusion` can help, but you may need to ensure fusion targets the im2col producer).
- Add a dedicated “im2col+matmul fused” macro-action:
 - convert conv2d $\rightarrow$ img2col + matmul,
 - tile matmul outer loops,
 - fuse img2col into matmul tiles,
 - then pack/promote/vectorize the matmul.

(2) Direct conv pipeline (avoid im2col when it's bad)

Experts often prefer direct conv for many shapes (esp when im2col materialization is expensive).

MLIR already has building blocks:

- Vectorization support for NHWC/HWCF conv ops ([reviews.llvm.org](#))
- A tiling pass for convolutions leveraging matmul infrastructure ([reviews.llvm.org](#))

What you likely don't have in your action space:

- Choosing between **direct conv** and **im2col** as an algorithmic action.
- Layout conversion / channel blocking actions (e.g., channels-last, channel blocking).
 - IREE explicitly provides preprocessing passes for “convert conv to channels last” and “convert conv2d to im2col” ([iree.dev](#)).

Practical RL integration:

- Add an action “ConvAlgoChoice”:
 - direct conv vs im2col.
 - Add an action “ConvLayoutChoice”:
 - convert NCHW $\leftrightarrow$ NHWC, choose filter layout (HWCF/FHWC), channel blocking factor (vector width).
-

6.3 Missing for Add (elementwise)

Your system *should* be able to get close, but common missing pieces are:

(1) Parallelism threshold + thread policy

Add is memory-bound; parallelizing too aggressively can hurt.

Implementation:

- This is mostly **configuration / runtime policy**, not a Linalg transform:
 - set a threshold: “only use TP if total elements > X”
 - optionally choose `num_threads = min(max_threads, total_work / grain)`
- In IR, you can also thread-guard parallel regions with runtime checks if you want full generality.

(2) Thread affinity / pinning

Configuration (not MLIR IR):

- Use OpenMP affinity env vars (varies by runtime), e.g. `OMP_PROC_BIND`, `OMP_PLACES`, etc.
- Ensure you’re not oversubscribing (watch for nested parallelism).

(3) Vector-width shaping without huge vectors

Instead of vectorizing “the whole tile” (which leads to giant vectors and unrolling), you want vector-level tiling:

- Tile to a fixed SIMD width, then vectorize.

This is the same missing “tile-to-vector-size” concept that matters for GEMM too; it’s explicitly part of expert MLIR CPU codegen pipelines (e.g., IREE’s LLVMCPU pipeline includes “tile-to-vector-size” and “peel” as named stages) .

7) “Expert MLIR recipes” you can copy

7.1 GEMM: pack + microkernel style (IREE / MLIR ecosystem)

If your goal is “copy something known-good”, the most actionable artifact is:

- **IREE’s pack/unpack + mmt4d integration test / recipe**, which demonstrates a realistic transform-based pipeline for matmul-style codegen (packing, lowering pack, vector lowering steps, etc.)

- Background on why `linalg.mmt4d` exists and how it maps to microkernels .

Even if you don't adopt IREE as a framework, the *transform scripts* and the *sequence of steps* are directly transferable to an MLIR+Transform setup.

7.2 Conv in MLIR

For direct conv:

- Use MLIR's conv tiling infrastructure (reviews.llvm.org) plus NHWC/HWCF vectorization support (reviews.llvm.org).

For im2col:

- MLIR has a canonical transformation that rewrites conv2d to `linalg.generic` (img2col packing) + `linalg.matmul` ([MLIR](https://reviews.llvm.org)), and there is a transform op specifically for conv2d→im2col (reviews.llvm.org).
- Be aware of the known pitfall: img2col can be hard to vectorize and may dominate if not fused or specialized ([GitHub](https://github.com)).

7.3 “Cheat” option: lower GEMM to a BLAS call

If the goal is simply “match PyTorch”, calling an optimized library is often the fastest path:

- MLIR has a “linalg to library call” lowering approach (e.g., `matmul` → `cbblas_*gemm`) as shown in MLIR tests .
- This aligns with Linalg's design intent: lowering to library calls / special instructions is a first-class route ([MLIR](https://reviews.llvm.org)).

8) Final checklist: missing transformations/configurations + how to implement

Below is the consolidated list you asked for.

Missing #1 — Packing / blocked layout (GEMM, Conv, sometimes Add epilques)

What: pack operands (esp B), optionally transpose/pack, pad to vector multiples.

MLIR implementation:

- `transform.structured.pack`

- `transform.structured.pack_greedily`
 - `transform.structured.lower_pack` (commonly used in full pipelines)
Where to add: new RL action (Pack) or macro-action before vectorization.
-

Missing #2 — Promotion to fast buffers (scratchpad/stack)

What: promote subviews/tiles of operands to temporary buffers, aligned, full-tile fast path.

MLIR implementation:

- `transform.structured.promote` (with `operands_to_promote`, `use_full_tile_buffers`, `alignment`)
 - Backed by Linalg promoteSubViews concepts ([MLIR](#))
Where to add: new RL action (Promote) parameterized by operands/alignment.
-

Missing #3 — Register tiling / microkernel shaping (MR×NR×KR)

What: a dedicated inner tiling level chosen for SIMD registers + reduction unrolling.

MLIR implementation:

- Implement as a **macro-action**: (tile-to-register-shape → vectorize → vector.contract lowering).
 - Optionally follow the `mmt4d` microkernel style used in practice .
Where to add: either expand tiling action to support a *second parameter set* “register tile sizes”, or add a macro-action that applies two tilings.
-

Missing #4 — Explicit unroll factors (and unroll-and-jam)

What: control K-unroll (KR) and sometimes unroll-and-jam.

MLIR implementation:

- `transform.loop.unroll` (factor)
 - `transform.loop.unroll_and_jam`
Where to add: new RL action(s) that select factors.
-

Missing #5 — Full/partial tile separation for general tiling (tails)

What: allow non-divisor tile sizes and handle tails via peel/pad/mask.

MLIR implementation:

- `transform.loop.peel` for remainder separation
- Use padding via pack (pack supports padding values)
- Pad vectorization support exists (tensor.pad vectorization patterns) ([MLIR](#))

Where to add: relax tile candidate constraints + add “peel/pad” follow-up actions.

Missing #6 — Vector-level tiling (“tile-to-vector-size”) as a first-class step

What: avoid “giant vectors + full unroll” by tiling to SIMD widths before vectorizing.

MLIR implementation:

- Do a dedicated tiling step to small static sizes (SIMD width), then `transform.structured.vectorize`.
- This is a standard stage in expert pipelines like IREE LLVMCPU (“tile-to-vector-size”, “peel”, etc.)

Where to add: bake into your `Vectorization` action (auto tile-to-vector-size) or add as a separate action.

Missing #7 — Conv algorithm + layout choice (direct vs im2col; NHWC/HWCF; channels-last; channel blocking)

What: choose the conv lowering strategy and data layout.

MLIR implementation options:

- Direct conv:
 - leverage MLIR’s conv tiling infrastructure ([reviews.llvm.org](#))
 - NHWC/HWCF conv vectorization support ([reviews.llvm.org](#))
- im2col:
 - `populateConvertConv2DToImg2ColPatterns` / rewrite to generic + matmul ([MLIR](#))
 - transform op exists for conv2d→im2col ([reviews.llvm.org](#))
- Layout passes (if you’re willing to borrow from IREE preprocessing ideas): conv to channels-last, conv2d→img2col ([iree.dev](#))

Where to add: new RL action(s) (ConvAlgoChoice, ConvLayoutChoice).

Missing #8 — Thread/runtime controls (add, gemm, conv)